

Constrained Policy Learning with Predictive Action Projection

Implementation Design

Salvador Tenorio

Abstract

This document specifies the implemented mathematical, software, and artifact contracts of a compact continuous-control reinforcement-learning study. The repository combines a fixed-shape Gymnasium navigation environment, a CleanRL-style PPO implementation, normalized policy actions, a softened control-barrier-function quadratic-program projection layer, reproducible training variants, single and paired evaluation, named layout suites, trajectory audit archives, Python-based orchestration, and metadata-driven result generation. The design records the ownership boundaries and invariants that must remain stable while the final experiments are conducted. It describes implemented behavior only; empirical conclusions belong to the experiment records and paper.

Contents

1	Purpose, authority, and scope	3
2	Repository architecture	3
2.1	Component architecture	3
2.2	Module map	5
2.3	Ownership invariants	6
3	Environment and task contract	6
3.1	State, obstacle capacity, and observation	6
3.2	Physical actions and dynamics	7
3.3	Collision clearance, success, and reward	7
3.4	Training layout and named evaluation suites	7
4	Action interfaces and wrapper order	8
4.1	Construction order	8
4.2	Normalized and physical actions	9
5	Predictive action projection	9
5.1	Lookahead geometry	9
5.2	Barrier constraints and QP	9
5.3	Special paths and failure semantics	10
5.4	Diagnostics	10

6	PPO implementation and training variants	11
6.1	Actor-critic	11
6.2	Rollout semantics	11
6.3	Training variants	11
6.4	Checkpoint and TensorBoard contract	11
7	Evaluation, layout suites, and trajectories	12
7.1	Evaluation modes	12
7.2	Episode result schema	13
7.3	Trajectory archive	13
8	Orchestration, validation, and calibration	13
8.1	Python orchestration and Windows launchers	13
8.2	Runtime validation	14
8.3	Core-suite calibration	14
9	Analysis and paper artifact generation	14
10	Default configuration	15
11	Limitations, interpretation, and extension boundary	15
12	Change policy	16

1 Purpose, authority, and scope

This design is the authoritative implementation-facing description of the repository. It replaces earlier aspirational designs that mixed several future studies, obsolete file paths, and retired development-stage terminology. The codebase remains the source of truth when implementation and prose disagree; the document should then be corrected rather than used to justify obsolete behavior.

The repository implements a compact simulated research system. Its scope is constrained continuous-control learning and local predictive action projection. It does not include hardware, robotics middleware, rendering, embedded control, real-time deployment, or a global route planner. The scientific contribution is evaluated through saved checkpoints, episode tables, trajectories, solver diagnostics, and reproducible analysis artifacts.

Four invariants organize the implementation:

1. The environment is the only component that advances physical state.
2. The policy distribution owns the sampled normalized raw action and its log probability.
3. The optional projection layer maps a physical raw action to a physical executed action and reports the burden of that map.
4. Evaluation and analysis consume frozen artifacts and do not update policy parameters.

The PPO implementation follows the inspectable single-file style promoted by CleanRL [4] and uses PPO with generalized advantage estimation [6, 5]. The projection layer is a small CBF-QP related to standard barrier-filter constructions [1], expressed in CVXPY [2], and solved with OSQP [7].

2 Repository architecture

2.1 Component architecture

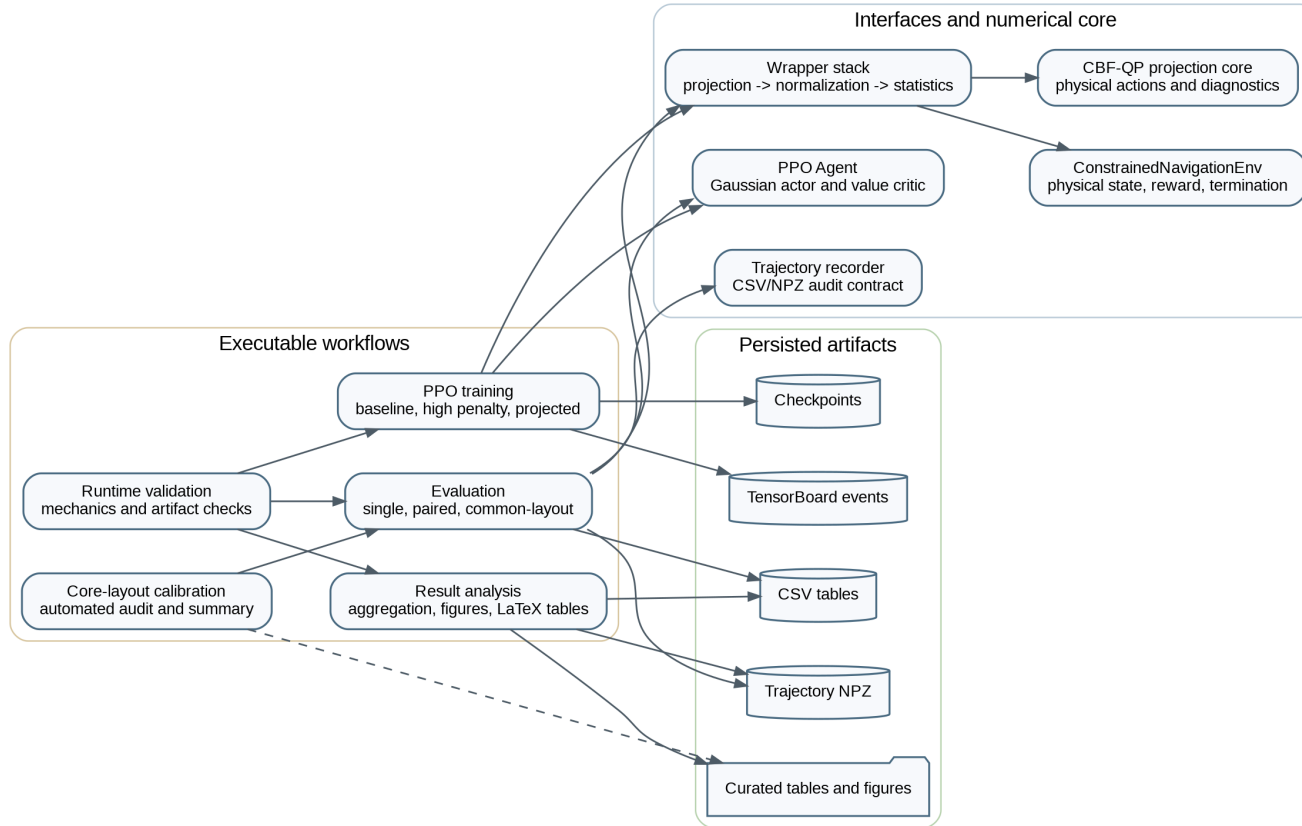


Figure 1: Current repository architecture. Training, evaluation, calibration, validation, and analysis reuse a small set of numerical components and produce traceable research artifacts.

The repository uses classes only where a stable framework or state boundary exists. The Gymnasium environment, action wrappers, actor-critic, and trajectory data object are natural class boundaries. Numerical transforms, audits, aggregation, plotting, and orchestration remain explicit functions and scripts.

2.2 Module map

Table 1: Primary repository modules and responsibilities.

Path	Responsibility
<code>environments/constrained_navigation.py</code>	Physical-action Gymnasium environment, fixed observation encoding, default training layout, controlled reset layouts, reward, success, collision, truncation, and task diagnostics.
<code>environments/action_wrappers.py</code>	Affine mapping between normalized policy actions and bounded physical actions.
<code>environments/factory.py</code>	Fresh environment construction and the exact wrapper nesting order.
<code>projection/cbf_qp_projection.py</code>	Projection parameters, lookahead geometry, barrier rows, QP solve, clipping, fallback, and numerical result object.
<code>projection/cbf_qp_wrapper.py</code>	Gymnasium wrapper that reads current environment geometry, executes the projected action, and appends slot-aligned diagnostics.
<code>algorithms/ppo/agent.py</code>	Orthogonally initialized Gaussian actor and value critic.
<code>algorithms/ppo/ppo_continuous_action.py</code>	CleanRL-style rollout collection, same-step vector autoreset, GAE, PPO updates, TensorBoard logging, and checkpoint writing.
<code>algorithms/ppo/projection_training.py</code>	Rollout-level projection diagnostics and fail-fast solver handling during projected training.
<code>evaluation/evaluate_policy.py</code>	Random and PPO evaluation, checkpoint compatibility, episode metrics, CSV output, and optional trajectory recording.
<code>evaluation/evaluate_projection_pair.py</code>	Same-checkpoint, same-seed projection-off/on evaluation and paired comparison artifacts.
<code>evaluation/evaluate_layout_suite.py</code>	Named-layout evaluation with checkpoint, suite, method, seed, reward, and projection provenance.
<code>evaluation/layout_suite.py</code>	JSON suite loading, validation, padding, and canonical semantic hashing.
<code>evaluation/trajectory_recording.py</code>	Auditable NPZ trajectory contract with raw and executed actions and solver diagnostics.
<code>experiments/train_ppo_variant.py</code>	Shared orchestration for baseline, high-penalty, and projection-enabled training variants.

Path	Responsibility
<code>experiments/run_clean_ppo_baseline.py</code>	Historical clean baseline regression pipeline.
<code>experiments/calibrate_core_layouts.py</code>	Automated core-suite calibration, technical audit, summaries, and representative trajectories.
<code>analysis/aggregate_projection_results.py</code>	Metadata-driven dataset discovery, completeness checks, checkpoint and method aggregation, and LaTeX tables.
<code>analysis/plot_projection_results.py</code>	Figures from saved tables, trajectories, and available TensorBoard events.
<code>analysis/build_projection_results.py</code>	One-command result build and PASS summary.
<code>evaluation/validate_evaluation_time_projection.py</code>	Canonical evaluation-time projection regression.
<code>evaluation/validate_pre_experiment_codebase.py</code>	Consolidated pre-experiment training, evaluation, artifact, and result-pipeline validation.
<code>scripts/*.bat</code>	Thin Windows launchers that enter the repository root, invoke a Python module, and return its exit code.

2.3 Ownership invariants

The base environment owns all physical state and accepts physical actions. The projection wrapper may inspect public environment state needed for the local geometry, but it does not alter environment internals directly. The actor-critic does not depend on environment implementation details. Experiment and validation modules orchestrate existing public interfaces rather than duplicating algorithms.

3 Environment and task contract

3.1 State, obstacle capacity, and observation

The physical agent state is position $p_t = (x_t, y_t)^\top$ and heading θ_t . The goal is $p_g \in \mathbb{R}^2$. Obstacle slot i contains center o_i , physical radius r_i , and mask $m_i \in \{0, 1\}$. The agent has radius r_{agent} .

The observation has fixed dimension

$$d_o = 6 + 5N_{\text{max}}, \quad (1)$$

where N_{max} is the obstacle capacity. The first six entries are

$$[x_t, y_t, \cos \theta_t, \sin \theta_t, x_g - x_t, y_g - y_t]. \quad (2)$$

Each obstacle slot contributes relative position, signed collision clearance, physical radius, and active mask. Inactive slots are encoded as zeros. The default capacity is three, which yields a 21-dimensional observation and preserves compatibility with the historical baseline checkpoint.

The capacity and active count are distinct. Increasing `max_obstacles` changes the neural-network input dimension. Changing `num_active_obstacles` only changes which built-in slots are active.

3.2 Physical actions and dynamics

The base action is

$$u_t = [v_t, \omega_t]^\top, \quad 0 \leq v_t \leq v_{\max}, \quad -\omega_{\max} \leq \omega_t \leq \omega_{\max}. \quad (3)$$

The environment applies one explicit-Euler unicycle step using the heading at the beginning of the step:

$$x_{t+1} = x_t + v_t \cos \theta_t \Delta t, \quad (4)$$

$$y_{t+1} = y_t + v_t \sin \theta_t \Delta t, \quad (5)$$

$$\theta_{t+1} = \text{wrap}(\theta_t + \omega_t \Delta t). \quad (6)$$

The base environment clips the physical action to its declared bounds and rejects non-finite actions.

3.3 Collision clearance, success, and reward

The signed collision clearance for active obstacle i is

$$d_{i,t}^{\text{clear}} = \|p_t - o_i\|_2 - (r_i + r_{\text{agent}}). \quad (7)$$

Collision occurs when the minimum active clearance is non-positive. When no obstacle is active, minimum clearance is undefined and is reported as NaN; it is not replaced by a large artificial value.

Success occurs when $\|p_t - p_g\|_2 \leq r_g$. The implemented one-step reward is

$$r_t = c_p(d_{t-1}^{\text{goal}} - d_t^{\text{goal}}) - c_u(v_t^2 + \omega_t^2) - c_{\text{time}} + r_g^{\text{term}} \mathbf{1}_{\text{success}} - c_c \mathbf{1}_{\text{collision}}, \quad (8)$$

with an additional distance penalty when the episode reaches its time limit. The collision penalty is configurable so the high-penalty comparison can change only that reward term.

3.4 Training layout and named evaluation suites

The built-in training layout fixes start $[0, 0]$, heading 0, goal $[4, 0]$, and three obstacles at $[1.5, 0.5]$, $[2.5, -0.5]$, and $[3.2, 0.4]$ with radii 0.30, 0.30, and 0.25.

Controlled layouts are supplied through `reset(options=...)`. Named JSON suites are loaded by `evaluation/layout_suite.py`. The development suite exists for software checks. The 24-layout core suite is a fixed deterministic transfer and robustness suite; it changes obstacle geometry while holding the start, heading, goal, dynamics, reward, and episode limit constant. Its canonical identity is derived from a sorted canonical JSON representation rather than line-ending-sensitive file bytes.

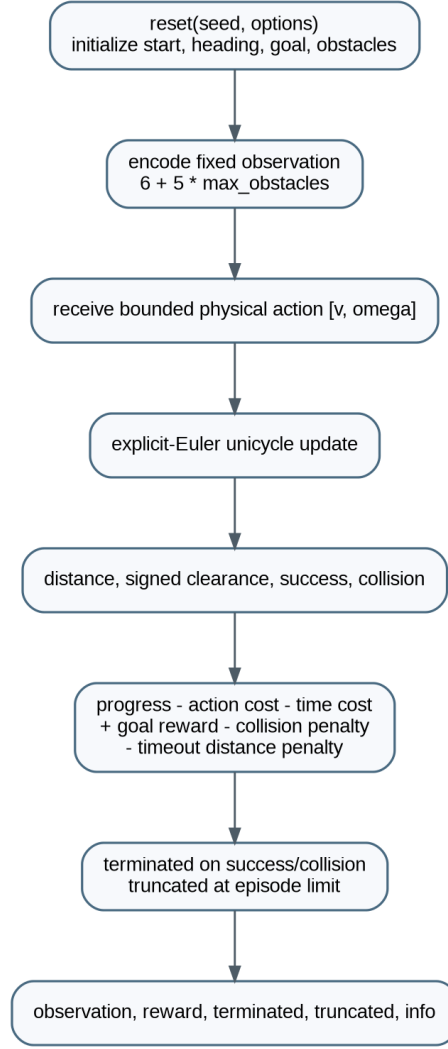


Figure 2: Environment reset and step contract.

4 Action interfaces and wrapper order

4.1 Construction order

The factory constructs the environment in this order:

$$\begin{aligned}
 \text{ConstrainedNavigationEnv} &\rightarrow \text{CbqPProjectionWrapper (optional)} \\
 &\rightarrow \text{NormalizedActionWrapper (optional)} \\
 &\rightarrow \text{RecordEpisodeStatistics (optional)}.
 \end{aligned}
 \tag{9}$$

Actions flow inward through the wrappers in reverse order.

4.2 Normalized and physical actions

The policy-facing action is $a^{\text{raw}} \in [-1, 1]^2$. The normalized wrapper maps it to the physical raw action

$$v = \frac{1}{2}v_{\max}(a_v + 1), \quad (10)$$

$$\omega = \omega_{\max}a_{\omega}. \quad (11)$$

Thus the actor mean $[0, 0]$ maps to physical action $[0.5v_{\max}, 0]$ rather than zero forward speed.

The physical raw action may then be projected:

$$u^{\text{exec}} = \begin{cases} u^{\text{raw}}, & \text{projection disabled,} \\ \Pi_{\text{safe}}(s, u^{\text{raw}}), & \text{projection enabled.} \end{cases} \quad (12)$$

The base environment receives only u^{exec} . PPO nevertheless stores and scores a^{raw} , the action sampled from its distribution.

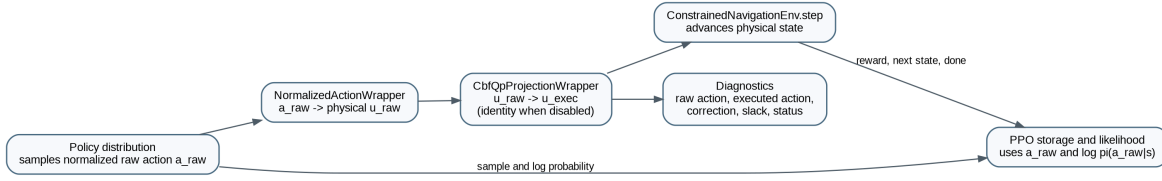


Figure 3: Normalized policy action, physical raw action, and physical executed action.

5 Predictive action projection

5.1 Lookahead geometry

The projection uses the lookahead point

$$p_L = p + L \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix} \quad (13)$$

and the action Jacobian

$$J_L(\theta) = \begin{bmatrix} \cos \theta & -L \sin \theta \\ \sin \theta & L \cos \theta \end{bmatrix}, \quad \dot{p}_L = J_L(\theta)u. \quad (14)$$

Collision geometry and projection conservatism are separate. The collision radius is $r_i + r_{\text{agent}}$. The projection radius is

$$R_i^{\text{proj}} = r_i + r_{\text{agent}} + r_{\text{extra}}, \quad (15)$$

where `extra_clearance` is a projection-only buffer.

5.2 Barrier constraints and QP

For active obstacle i ,

$$h_i(p_L) = \|p_L - o_i\|_2^2 - (R_i^{\text{proj}})^2, \quad \nabla h_i = 2(p_L - o_i). \quad (16)$$

The softened local condition is

$$\nabla h_i(p_L)^\top J_L(\theta)u + \xi_i \geq -\alpha h_i(p_L), \quad \xi_i \geq 0. \quad (17)$$

The executed action is the solution of

$$\min_{u, \xi} \frac{1}{2} \|u - u^{\text{raw}}\|_2^2 + \frac{\rho}{2} \|\xi\|_2^2 \quad (18)$$

subject to the CBF rows and physical action bounds.

5.3 Special paths and failure semantics

With no active constraints, the projector clips the raw physical action and returns success with zero slack. A successful QP returns the bounded action, finite slot-aligned slack values, and the solver status. A failed solve executes the stationary physical action $[0, 0]$, sets **success=False**, preserves a failure status, and reports active slack as unknown rather than as valid zero.

5.4 Diagnostics

The numerical result and wrapper information include raw action, executed action, correction vector, correction norm, intervention flag, fixed-slot slack vector, summed slack, maximum slack, active-constraint count, solver status, success flag, and objective value. These fields are research measurements, not debugging leftovers.

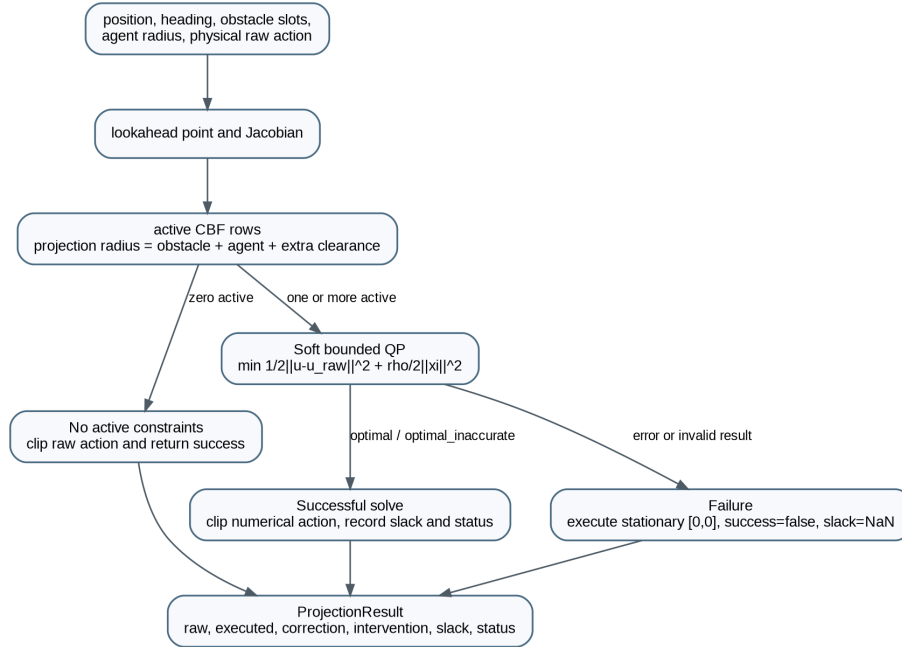


Figure 4: Projection geometry, optimization, and explicit failure path.

6 PPO implementation and training variants

6.1 Actor-critic

The actor and critic each use two 64-unit tanh hidden layers with orthogonal initialization. The actor outputs the mean of a diagonal Gaussian distribution and owns a learned log-standard-deviation parameter. The critic outputs one value estimate per observation.

6.2 Rollout semantics

The trainer uses `gym.vector.SyncVectorEnv` with `AutoresetMode.SAME_STEP`. Every sampled action therefore corresponds to an executed transition; terminal episode data are preserved under `final_info`, while the next observation is already the reset state of the next episode.

At rollout step t , PPO stores

$$(s_t, a_t^{\text{raw}}, \log \pi_{\theta_{\text{old}}}(a_t^{\text{raw}}|s_t), r_t, d_t, V_t). \quad (19)$$

The wrapper stack transforms a_t^{raw} into u_t^{exec} before the environment transition. The PPO ratio is still computed from the stored normalized raw action:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t^{\text{raw}}|s_t)}{\pi_{\theta_{\text{old}}}(a_t^{\text{raw}}|s_t)}. \quad (20)$$

Rewards and next states arise from u_t^{exec} . This is an empirical projected-environment formulation, not a corrected unbiased projected policy-gradient method [3].

6.3 Training variants

The shared Python launcher supports three declared methods:

Table 2: Implemented training variants.

Method label	Collision penalty	Training projection	Purpose
<code>ppo_baseline</code>	10	disabled	Reference PPO training.
<code>ppo_high_penalty</code>	50	disabled	Reward-shaping comparison.
<code>ppo_train_projection</code>	10	enabled	Policy adaptation under projected transitions.

The Windows batch files are thin launchers around `experiments/train_ppo_variant.py`. Projection-enabled training accumulates rollout-level intervention frequency, mean correction norm, mean summed slack, and maximum slack. Any reported projection failure aborts training rather than silently entering the final dataset.

6.4 Checkpoint and TensorBoard contract

A checkpoint contains the agent state, complete CLI argument mapping, run name, global step, observation and action shapes, and flattened dimensions. TensorBoard records PPO losses, learning

rate, throughput, completed-episode return and length, terminal success/collision, terminal clearance when defined, and projection burden for projected training.

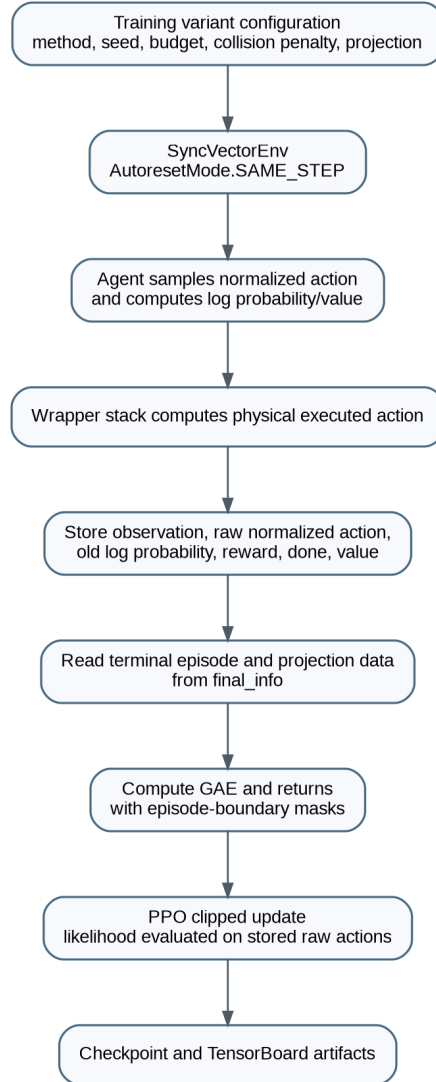


Figure 5: Implemented PPO training path with same-step autoreset and optional projection.

7 Evaluation, layout suites, and trajectories

7.1 Evaluation modes

`evaluation.evaluate_policy` evaluates random actions or a PPO checkpoint. PPO evaluation may use the deterministic actor mean or stochastic action samples. Deterministic evaluation supports repeatable task and transfer comparisons. Stochastic evaluation exposes action-distribution safety behavior and is used for active projection diagnostics.

`evaluation.evaluate_projection_pair` loads one checkpoint and runs projection-disabled and projection-enabled modes with the same seeds and environment configuration. It writes two

raw episode CSVs, two trajectory archives, one paired episode table, and one paired summary table.

`evaluation.evaluate_layout_suite` applies one checkpoint to every named layout in a validated JSON suite. Each row records the method, training seed, checkpoint path and hash, suite ID and hash, layout ID, repeat, evaluation seed, reward settings, projection mode, and episode metrics. Existing output files are refused unless replacement is explicitly requested.

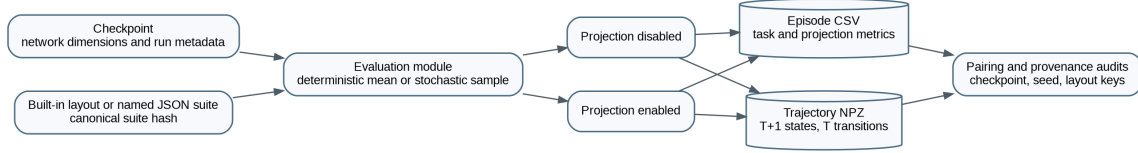


Figure 6: Evaluation modes and their auditable CSV and NPZ products.

7.2 Episode result schema

Every episode row contains policy identity, seed, return, length, success, collision, final distance, minimum obstacle clearance, projection mode, intervention count and rate, mean and maximum correction, mean summed slack, maximum individual slack, and solver-failure count. Projection-specific values remain in a fixed schema even when projection is disabled.

Failed solves invalidate episode slack statistics: unknown slack remains `NaN` and is not converted to zero. Successful zero-slack steps remain true zeros and are included in means.

7.3 Trajectory archive

The archive version is `evaluation_trajectory_v1`. It contains only non-object NumPy arrays and is loaded with `allow_pickle=False`. For an episode with T transitions, state arrays have $T + 1$ samples and transition arrays have T samples. Index t therefore aligns

$$s_t, a_t^{\text{raw}}, u_t^{\text{raw}}, u_t^{\text{exec}}, r_t, s_{t+1}. \quad (21)$$

The archive stores positions, headings, step counts, goal distance, clearance, success/collision state, normalized raw actions, physical raw actions, physical executed actions, corrections, rewards, terminal flags, projection flags, slot-aligned slack, solver status, and static geometry.

8 Orchestration, validation, and calibration

8.1 Python orchestration and Windows launchers

Substantive workflow logic lives in Python. Batch files exist only to provide convenient Windows entry points, change to the repository root, invoke a Python module, and propagate the exit code. This boundary avoids complex shell logic while preserving explicit runnable commands.

The orchestration modules validate paths before deletion or replacement, use the active Python interpreter for child commands, write structured summaries, and create PASS markers only after every required phase succeeds.

8.2 Runtime validation

The canonical evaluation-time validator checks environment and solver availability, checkpoint identity, active source compilation, the lightweight test suite, observation-capacity compatibility, expected incompatible-capacity rejection, deterministic paired evaluation, stochastic active projection, CSV and trajectory identities, solver failures, and a repeated QP runtime benchmark.

The pre-experiment validator extends that path with short baseline, high-penalty, and projected training runs; checkpoint and TensorBoard audits; development-layout evaluation; result aggregation; figure generation; and concise manual-review instructions.

8.3 Core-suite calibration

The automated calibration evaluates a historical regression checkpoint on the 24-layout core suite with projection disabled and enabled. It verifies identities, aligned episode keys, finite metrics, action and trajectory contracts, intervention diversity, and zero solver failures. It generates family and layout tables and four predeclared representative trajectory panels. A focused manual judgment then decides whether the geometry is credible. The historical checkpoint is a probe, not a final replicate.

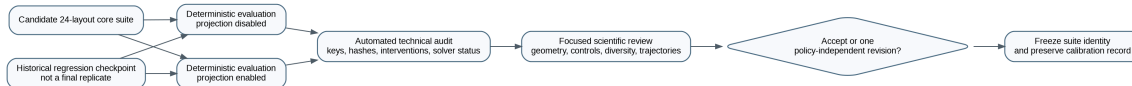


Figure 7: Core-suite calibration and freeze workflow.

9 Analysis and paper artifact generation

The analysis protocol declares expected methods, training seeds, required projection modes, layout suite, evaluation reward, projection parameters, representative layout, and whether complete paper evidence is mandatory.

The aggregator discovers evaluation CSVs by schema and internal metadata rather than by hard-coded generated filenames. It rejects duplicate keys, incomplete method/seed/layout coverage, mismatched suite or checkpoint identities, non-finite results, and nonzero solver failures. It first averages layouts within each checkpoint and only then aggregates independently trained checkpoints across seeds. This prevents layout-level pseudoreplication.

The plotter consumes audited tables, trajectory archives, and available TensorBoard event data. Final protocols require complete training-return and representative-trajectory evidence for every declared method and mode; the development protocol permits optional omissions for smoke validation. Result output directories must be empty to prevent stale figures from being mistaken for current artifacts.

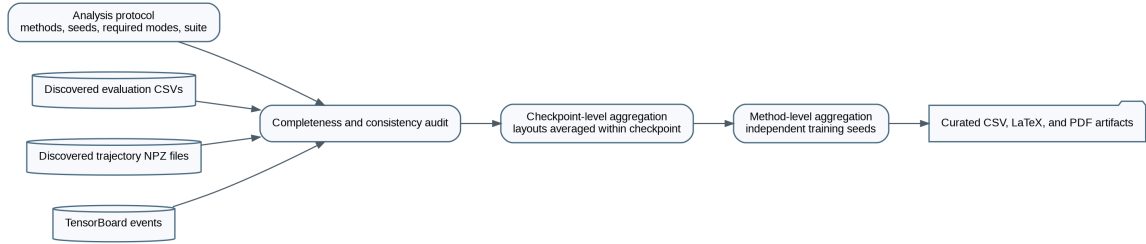


Figure 8: Artifact discovery, audit, hierarchical aggregation, and paper-ready output.

10 Default configuration

Table 3: Implemented defaults and fixed study values.

Parameter	Value	Role
<code>max_obstacles</code>	3	Fixed observation capacity.
<code>num_active_obstacles</code>	3	Active built-in training obstacles.
<code>dt</code>	0.1	Simulation step.
<code>v_max</code>	1.0	Maximum forward speed.
<code>omega_max</code>	2.0	Maximum turn-rate magnitude.
<code>goal_radius</code>	0.25	Success radius.
<code>agent_radius</code>	0.10	Collision footprint.
<code>max_episode_steps</code>	200	Time-limit truncation.
<code>progress_weight</code>	1.0	Progress reward weight.
<code>action_penalty</code>	0.01	Quadratic physical-action cost.
<code>time_penalty</code>	0.01	Per-step cost.
<code>goal_reward</code>	10.0	Terminal success reward.
<code>collision_penalty</code>	10.0	Default collision cost.
High-penalty comparison	50.0	Explicit reward-shaping variant.
<code>timeout_distance_penalty</code>	1.0	Distance penalty at truncation.
<code>lookahead_distance</code>	0.25	Projection lookahead.
<code>alpha</code>	2.0	Barrier-rate parameter.
<code>extra_clearance</code>	0.0	Projection-only safety buffer.
<code>slack_penalty</code>	1000.0	Quadratic slack weight.
<code>correction_tolerance</code>	10^{-6}	Intervention threshold.
Core training budget	51,200 transitions	Planned final comparison budget.
Core training seeds	1, 2, 3	Independent trained-policy replicates.

11 Limitations, interpretation, and extension boundary

The benchmark is deliberately compact. Training uses one deterministic three-obstacle layout. The frozen 24-layout suite measures deterministic transfer across obstacle geometries rather than

broad task-distribution generalization. There is no workspace boundary, so a poor policy may travel far beyond the goal. The projection condition is local, softened by slack, and evaluated in a discrete-time simulator; it is not a global safety guarantee or route planner.

Projection-enabled PPO uses raw-action likelihoods and projected transitions. High intervention burden can therefore indicate that safety is supplied primarily by the filter. Projection can prevent a collision without producing a successful route, and persistent unsafe policy behavior can exceed what a local filter can recover from.

The current environment, checkpoints, trajectory archives, and evaluation contracts are suitable bases for later artifact-consuming studies. New structured fitting or offline-learning modules should be additive consumers of exported states, actions, values, returns, and transitions. They should not require rewriting the environment, action map, projection core, or current evaluation schemas.

12 Change policy

During the final experiment campaign, code should remain frozen unless a concrete validity-threatening defect appears. Changes must preserve, or explicitly revise, the following contracts:

- observation capacity determines checkpoint input dimension;
- normalized action $[0, 0]$ maps to physical $[0.5, 0]$ under default limits;
- projection is inside the normalized action wrapper;
- collision geometry and projection-only extra clearance remain distinct;
- failed projection executes $[0, 0]$ and reports unknown slack;
- PPO likelihoods remain attached to normalized raw actions;
- layout suite identity remains semantic and line-ending independent;
- final aggregation averages layouts within checkpoint before aggregating training seeds;
- plotting and tables are generated from saved artifacts only.

References

- [1] Aaron D. Ames et al. “Control Barrier Function Based Quadratic Programs for Safety Critical Systems”. In: *IEEE Transactions on Automatic Control* 62.8 (2017), pp. 3861–3876. DOI: 10.1109/TAC.2016.2638961.
- [2] Steven Diamond and Stephen Boyd. “CVXPY: A Python-Embedded Modeling Language for Convex Optimization”. In: *Journal of Machine Learning Research* 17.83 (2016), pp. 1–5. URL: <https://www.jmlr.org/papers/v17/15-408.html>.
- [3] Sébastien Gros, Mario Zanon, and Alberto Bemporad. “Safe Reinforcement Learning via Projection on a Safe Set: How to Achieve Optimality?” In: *IFAC-PapersOnLine* 53.2 (2020), pp. 8076–8081. DOI: 10.1016/j.ifacol.2020.12.2276.
- [4] Shengyi Huang et al. “CleanRL: High-Quality Single-File Implementations of Deep Reinforcement Learning Algorithms”. In: *Journal of Machine Learning Research* 23.274 (2022), pp. 1–18. URL: <https://www.jmlr.org/papers/v23/21-1342.html>.
- [5] John Schulman et al. “High-Dimensional Continuous Control Using Generalized Advantage Estimation”. In: *International Conference on Learning Representations*. 2016. arXiv: 1506.02438.
- [6] John Schulman et al. “Proximal Policy Optimization Algorithms”. In: *arXiv preprint arXiv:1707.06347* (2017). DOI: 10.48550/arXiv.1707.06347.
- [7] Bartolomeo Stellato et al. “OSQP: An Operator Splitting Solver for Quadratic Programs”. In: *Mathematical Programming Computation* 12.4 (2020), pp. 637–672. DOI: 10.1007/s12532-020-00179-2.